News in EcmaScript 5/6

MARTIN PETÝREK

http://petyrek.github.io/

EcmaScript 5

Strict mode

‘use strict’;

- First line of you file or inside <script> element
- Can be switched on per function
- </>

Data properties

- Writable
- Enumerable
- Configurable

Accessor properties

- Get and set methods for properties
- </>

Metaprogramming

Getting and setting prototypes

- Object.create();
- Object.getPrototypeOf();

Metaprogramming

Managing property attributes via property descriptors

- Object.defineProperty();
- Object.defineProperties();
- Object.getOwnPropertyDescriptor();

Metaprogramming

Listing properties

- Object.keys();
- Object.getOwnPropertyNames();

Metaprogramming

Protecting objects

- Object.preventExtensions();
- Object.isExtensible();
- Object.seal();
- Object.isSealed();
- Object.freeze();
- Object.isFrozen();

Function prototype functions

Function.prototype.bind(thisValue, arg1, ..., argN)

- </>

String prototype functions

- access characters via bracket operator [...]
- String.prototype.trim()

Array prototype functions

- Array.isArray()
- Array.prototype.every()
- Array.prototype.filter()
- Array.prototype.forEach()
- Array.prototype.indexOf()
- Array.prototype.lastIndexOf()
- Array.prototype.map()
- Array.prototype.reduce()
- Array.prototype.some()

JSON support

- JSON.parse()
- JSON.stringify()

EcmaScript 6

Arrow functions

- Syntactic sugar for anonymous functions
- Share `this` with surrounding code

```
var numbers = [1,2,3,4,5];
numbers.forEach(x => {
        console.log(x);
});
```

Classes

- Syntactic sugar for prototype based OO pattern
- Key words like: class, super, extends, constructor

```
class Rectangle extends Shape{
        constructor(x, y, width, height){
                super(x, y);
        }
}
```

Template strings

- Enclosed in ``
- Expression interpolation

```
`Hello ${name}`;

`Today is ${new Date()}`;
```

Destructuring

- list matching, equivalent code

```
var [m, d, y] = [3, 14, 1977];

var m = 3, d = 14, y = 1977;
```

Default function parameters

```
function foo(x, y=12){
        return x + y;
}
```

Rest function parameters

- If last argument is preceeded with … it becomes an array

```
function f(x, ...y) {
return x * y.length;
}
f(3, "hello", true) == 6
```

Spread operator

- If argument is preceeded with …

```
function f(x, y, z) {
        return x + y + z;
}

f(...[1,2,3]) == 6
```

let keyword

- Block scope variable declaration

```
let x = 5;
```

const keyword

- Creates read-only block scoped constant

```
const x = 5;
```

for .. of loop

```
let arr = [3, 5, 7];
for (let i of arr) {
  console.log(i);
}
```

Generators

```
function* idMaker(){
  var index = 0;
  while(true)
    yield index++;
}

var gen = idMaker();
console.log(gen.next().value);
```

Unicode support

```
"\u{20BB7}"=="吉"=="\uD842\uDFB7"
```

Modules

```
export function sum(x, y) {
  return x + y;
}
```

```
import * as math from "lib/math";
alert("2π = " + math.sum(math.pi, math.pi));
```

Data structures

- Map
- Set
- WeakMap
- WeakSet

Proxying

```
var target = {};
var handler = {
  get: function (receiver, name) {
    return `Hello, ${name}!`;
  }
};
var p = new Proxy(target, handler);
p.world === 'Hello, world!';
```

Tail recursion

```
function factorial(n, acc = 1) {
        if (n <= 1) return acc;
        return factorial(n - 1, n * acc);
}
factorial(100000);
```

- Stack overflow in most implementations today,
- but safe on arbitrary inputs in ES6

Promises

- object used for deferred and asynchronous computations

```
getJSON("/api/employee/1").then(function(post) {
        return getJSON(post.commentURL);
});
```

Symbol

- New primitive type
- Used as unique IDs
- No two symbols are equal

```
let symbol = Symbol('symbol');
```

And many more…

- Standard library –
  - Additions to Array prototype
  - New math functions
  - Additions to the Number, Object, Regexp, String objects
- Typed arrays
- Binary and octal number literals

ES 6 today

- https://babeljs.io/docs/using-babel/
- Compile ES6 -> ES5 Strict
- Works with Node.js, Meteor, Rails, Grunt and other build systems

Thank you for your attention

Time for questions